

数据库系统课程设计报告

小组名称：邻安养老（待组员确认）

小组成员：待提供真实名单

撰写时间：2026 年 11 月 12 日

文档版本：V1.0

目录

一、成员分工.....3

二、需求分析.....4

三、系统概念结构设计.....11

四、系统的逻辑结构设计.....17

五、系统物理结构设计、持久层设计和关键业务代码.....22

六、项目运维管理和优化.....48

七、附件内容.....60

一、成员分工

项目题目为社区居家养老智慧服务平台的设计与实现。当前技术成果覆盖五条业务线，采用 Java 17 与 MySQL，提交材料需由实际组员核对、学习和演示。人员名单与实际贡献尚未提供，以下为可分配的责任范围，不作为已发生的分工记录。

责任范围	需要承担的数据库工作	协作关系
档案与评估	用例和鲁棒分析、局部模型、3 组双实现 SQL、视图、过程、触发器与测试	与公共模型和关联业务核对字段、状态及验收口径
下单与派单	用例和鲁棒分析、局部模型、3 组双实现 SQL、视图、过程、触发器与测试	与公共模型和关联业务核对字段、状态及验收口径
护工与排班	用例和鲁棒分析、局部模型、3 组双实现 SQL、视图、过程、触发器与测试	与公共模型和关联业务核对字段、状态及验收口径
监测与应急	用例和鲁棒分析、局部模型、3 组双实现 SQL、视图、过程、触发器与测试	与公共模型和关联业务核对字段、状态及验收口径
补贴与监管	用例和鲁棒分析、局部模型、3 组双实现 SQL、视图、过程、触发器与测试	与公共模型和关联业务核对字段、状态及验收口径

组长、4—5 名成员的姓名学号、每人的实际任务和协作情况：待团队填写。不得将上述建议责任范围直接改写为未经发生的个人贡献。

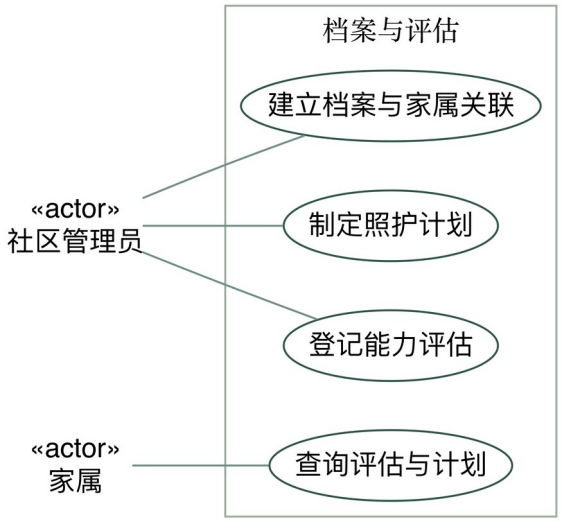
二、需求分析

平台围绕“建档—评估—计划—下单—派单—服务—工时—补贴—监管”组织数据。设备监测与应急处理通过老人档案关联主流程。核心问题包括有效期失效后仍接单、护工超量或重叠服务、改派历史被覆盖、设备换绑导致历史错归属和补贴缺乏服务依据。

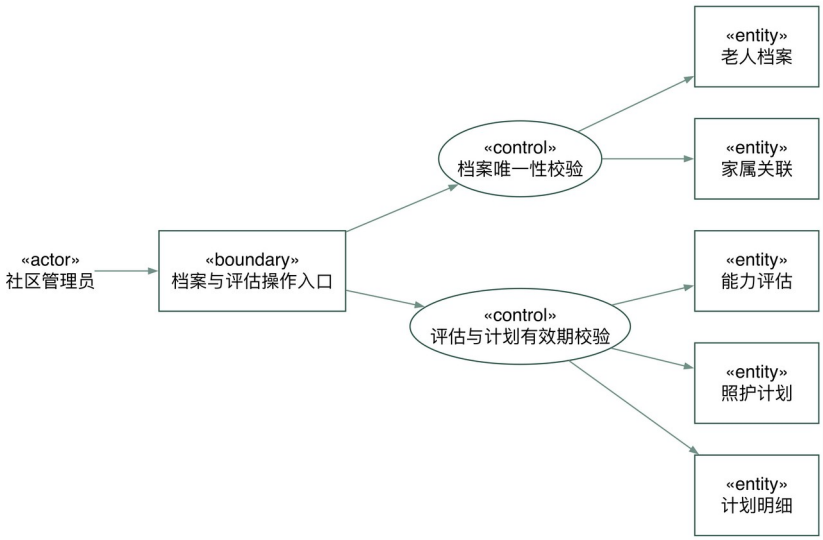
参与者包括社区管理员、老人及家属、护工、服务机构负责人、街道与民政监管。街道初审和民政终审在账号角色上细分。家属只访问关联老人，护工只访问本人订单，机构负责人只管理本机构护工，监管人员不能任意修改原始服务事实。

档案与评估

管理员建立唯一档案和家属关联，登记有效评估，再选择匹配能力等级的照护包并制定计划。每份计划引用采用的评估，系统检查有效期覆盖与区间重叠。



档案与评估用例图

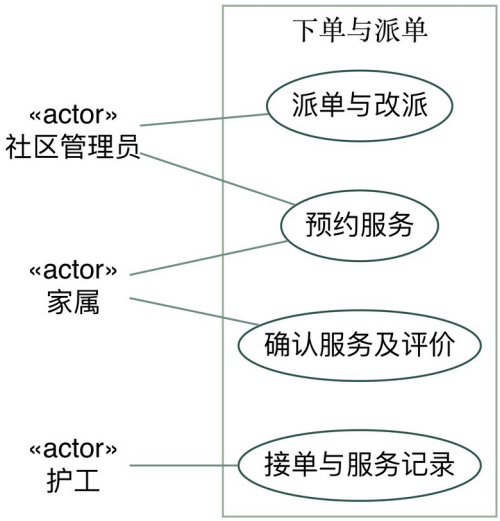


档案与评估鲁棒图

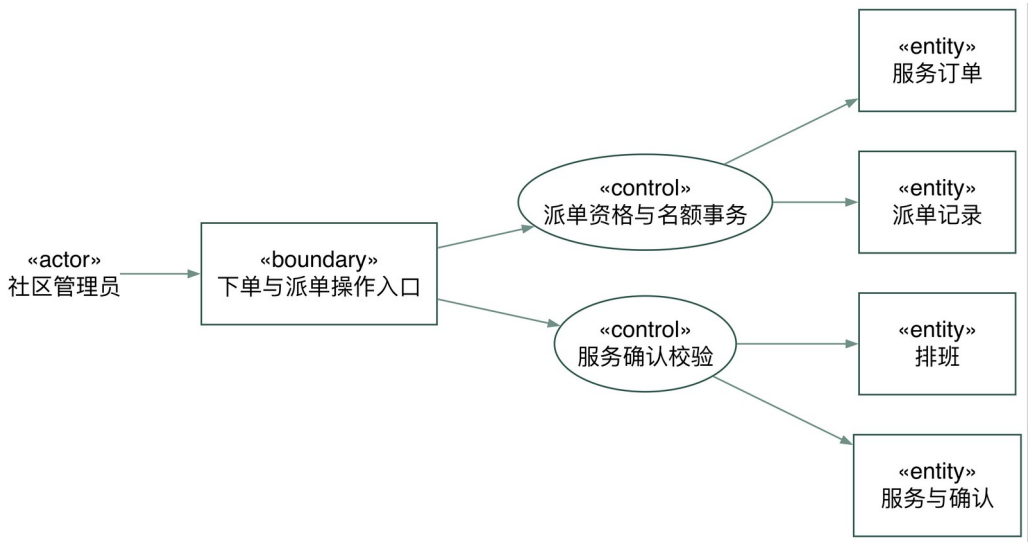
鲁棒图区分操作入口（boundary）、规则与事务控制（control）、持久化实体（entity）。参与者通过入口发起操作，控制对象校验业务条件后访问实体。主要数据项包括：老人档案、老人标签、家属关联、能力评估、照护服务包、服务包明细、照护计划、计划明细、账号。字段名称、粗略类型和说明见数据字典附件。

下单与派单

家属或社区创建预约。管理员按资质、区域、排班和整改状态派单。护工接单、签到、填写记录和签退后进入待确认，家属确认后订单才完成。改派记录保留，失败时恢复原分配。



下单与派单用例图

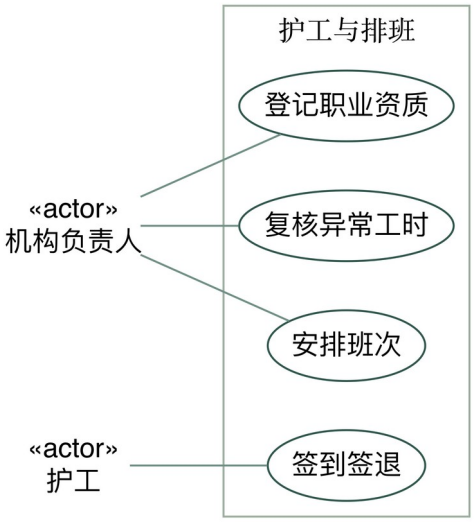


下单与派单鲁棒图

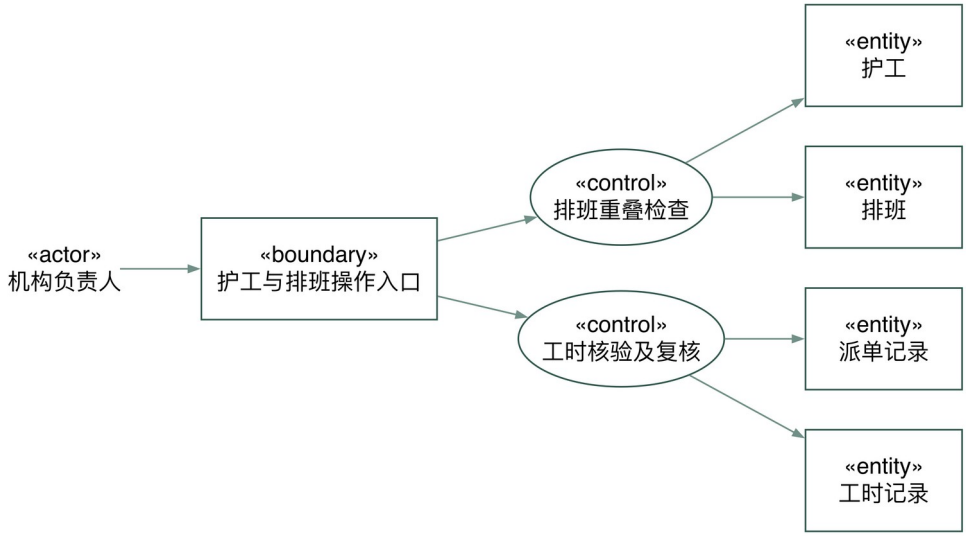
鲁棒图区分操作入口（boundary）、规则与事务控制（control）、持久化实体（entity）。参与者通过入口发起操作，控制对象校验业务条件后访问实体。主要数据项包括：老人档案、服务项目、服务订单、派单记录、服务与确认、评价投诉、排班。字段名称、粗略类型和说明见数据字典附件。

护工与排班

机构负责人登记项目资质与班次，护工服务前后打卡。容量不得超上限，班次和实际预约不得重叠。模拟围栏或时段异常进入人工复核，未经认定的工时不参与补贴。



护工与排班用例图

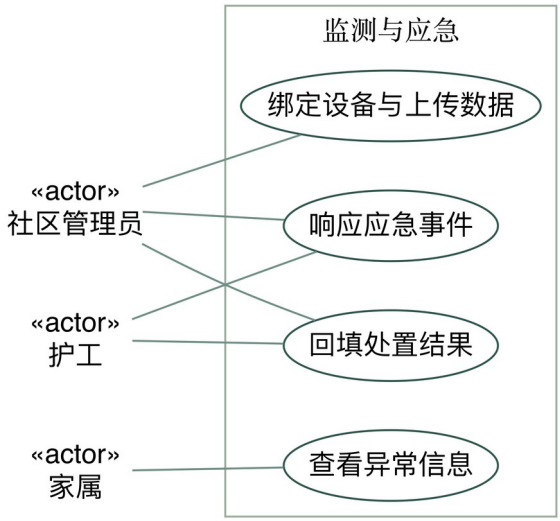


护工与排班鲁棒图

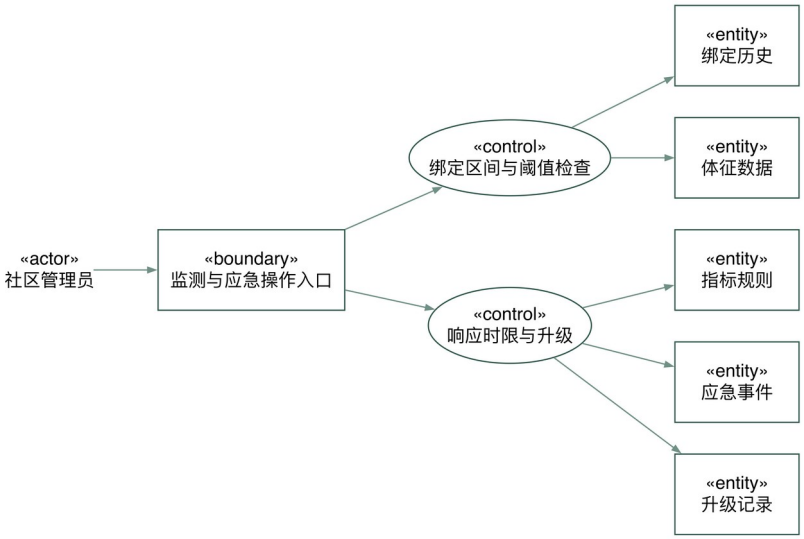
鲁棒图区分操作入口（boundary）、规则与事务控制（control）、持久化实体（entity）。参与者通过入口发起操作，控制对象校验业务条件后访问实体。主要数据项包括：服务机构、账号、护工、项目资质、护工服务区域、社区、排班、工时记录、派单记录。字段名称、粗略类型和说明见数据字典附件。

监测与应急

设备绑定历史保存老人归属和时间区间。体征上传先校验采集时绑定，再按课程演示阈值标记异常并建立待确认事件。人员响应后记录处置；超时通过定时任务升级并产生通知。



监测与应急用例图

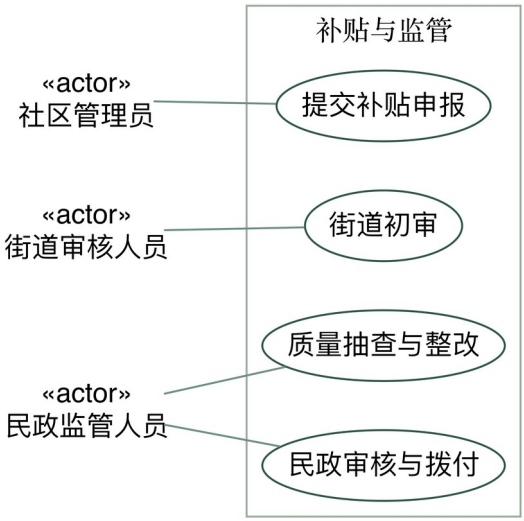


监测与应急鲁棒图

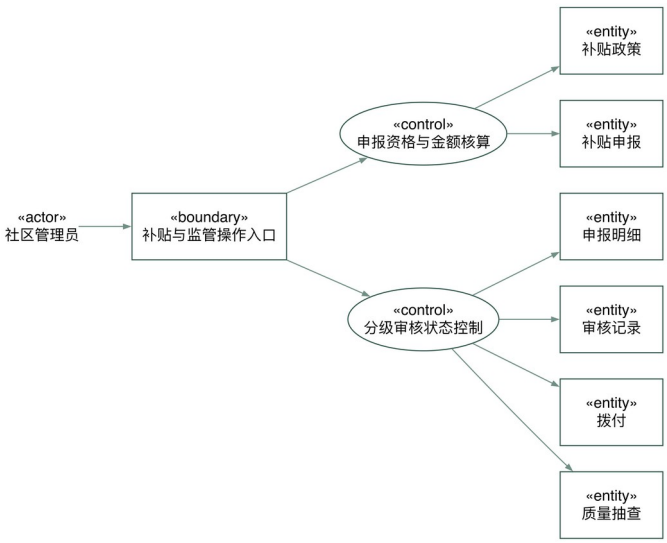
鲁棒图区分操作入口（boundary）、规则与事务控制（control）、持久化实体（entity）。参与者通过入口发起操作，控制对象校验业务条件后访问实体。主要数据项包括：老人档案、设备、绑定历史、指标规则、体征数据、应急事件、升级记录、通知。字段名称、粗略类型和说明见数据字典附件。

补贴与监管

社区按服务月份归集可补贴订单。申报事务逐笔检查确认、有效工时、政策和重复资格，任一不合格整体回滚。街道和民政分级审核，再登记模拟拨付；监管抽查和整改结果影响接单。



补贴与监管用例图



补贴与监管鲁棒图

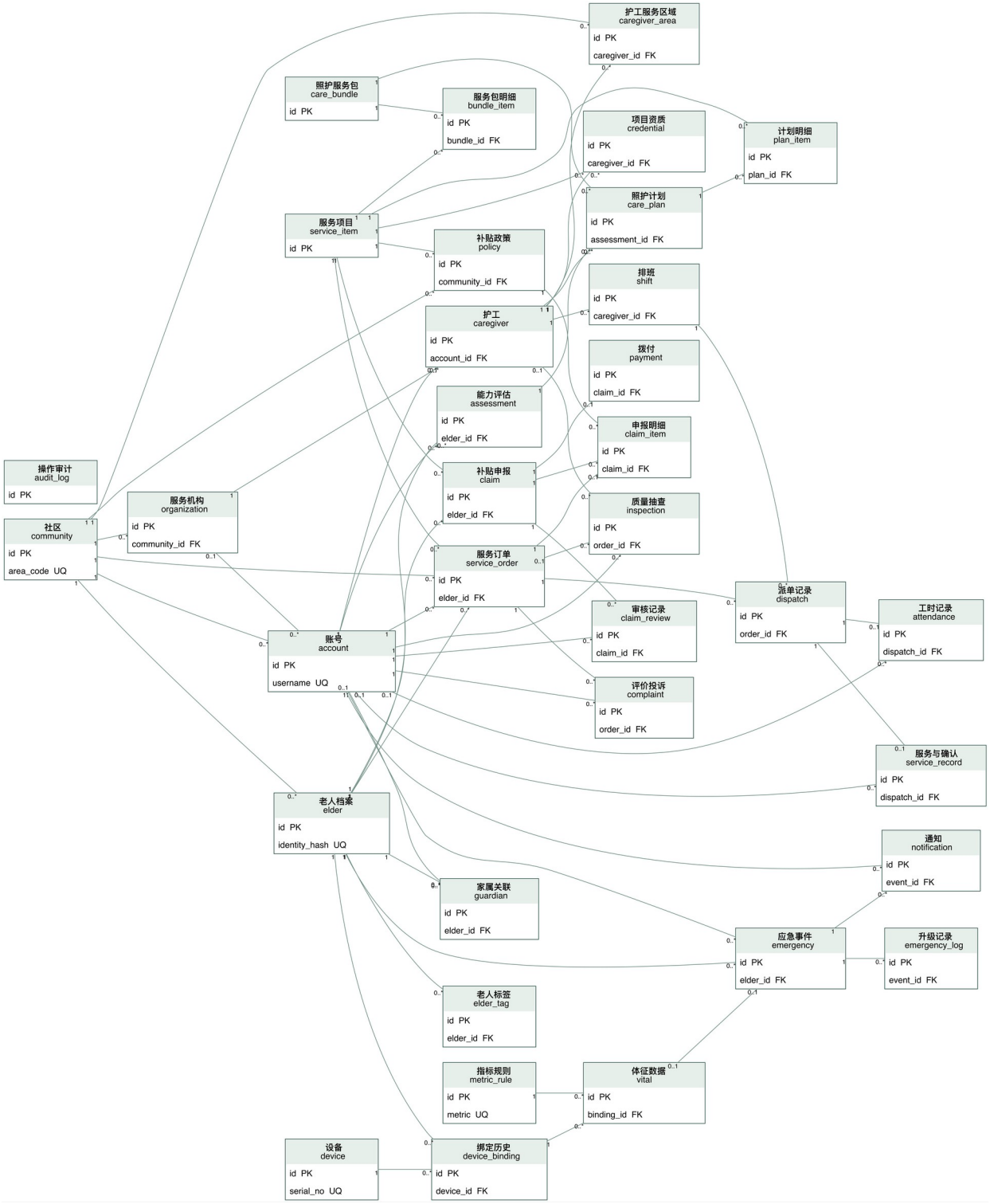
鲁棒图区分操作入口（boundary）、规则与事务控制（control）、持久化实体（entity）。参与者通过入口发起操作，控制对象校验业务条件后访问实体。主要数据项包括：老人档案、服务订单、补贴政策、补贴申报、申报明细、审核记录、拨付、质量抽查、账号。字段名称、粗略类型和说明见数据字典附件。

非功能性需求

完整性方面使用主外键、唯一、范围检查以及跨表业务校验。并发方面对同老人计划和同护工排班串行检查，对最后一个接单名额原子占用。安全方面实施密码摘要、失败锁定、会话到期、参数化 SQL、角色和行级过滤。历史方面保留改派、设备绑定、审核与审计记录。恢复方面采用全量备份结合 binlog，并在独立库验证。

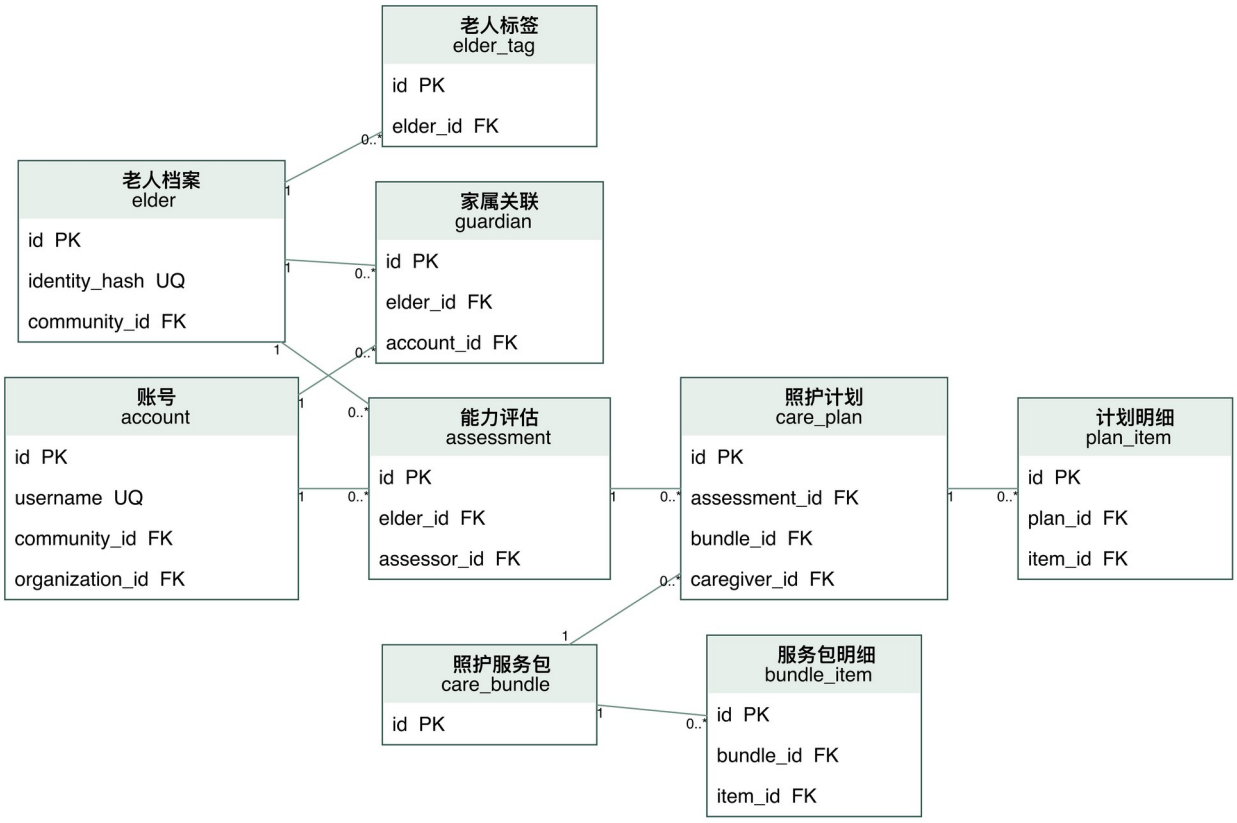
三、系统概念结构设计

全局概念模型包含 35 个持久化实体。老人、服务项目和账号贯穿多个业务域。下图展示全局联系概览，完整可缩放类图随附件提供；随后按业务给出局部模型，字段和联系以实际数据库为准。PK 为主键，FK 为外键，UQ 为唯一键。



全局 UML 类图概览，完整图见附件 01

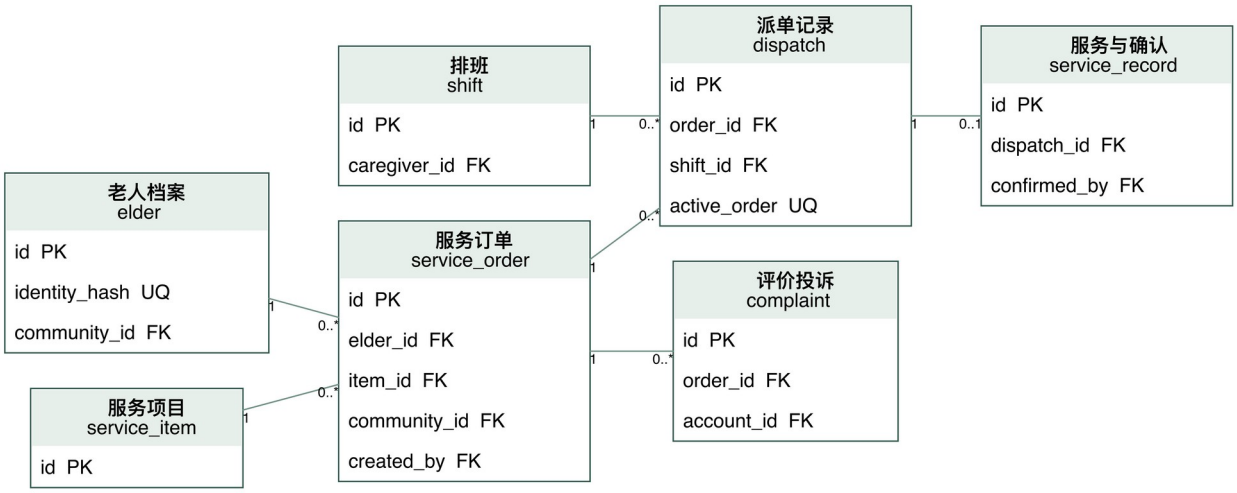
档案与评估局部模型



档案与评估局部 UML 类图

关联基数按业务含义和数据库外键标注。多个实体共用主数据时引用同一编号；可选关联使用允许空值的外键；一对多历史记录不通过覆盖旧行实现。

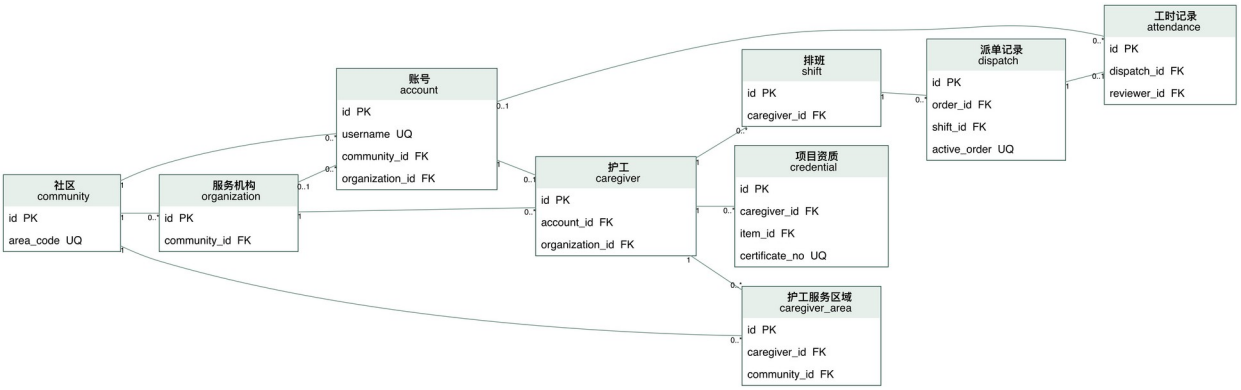
下单与派单局部模型



下单与派单局部 UML 类图

关联基数按业务含义和数据库外键标注。多个实体共用主数据时引用同一编号；可选关联使用允许空值的外键；一对多历史记录不通过覆盖旧行实现。

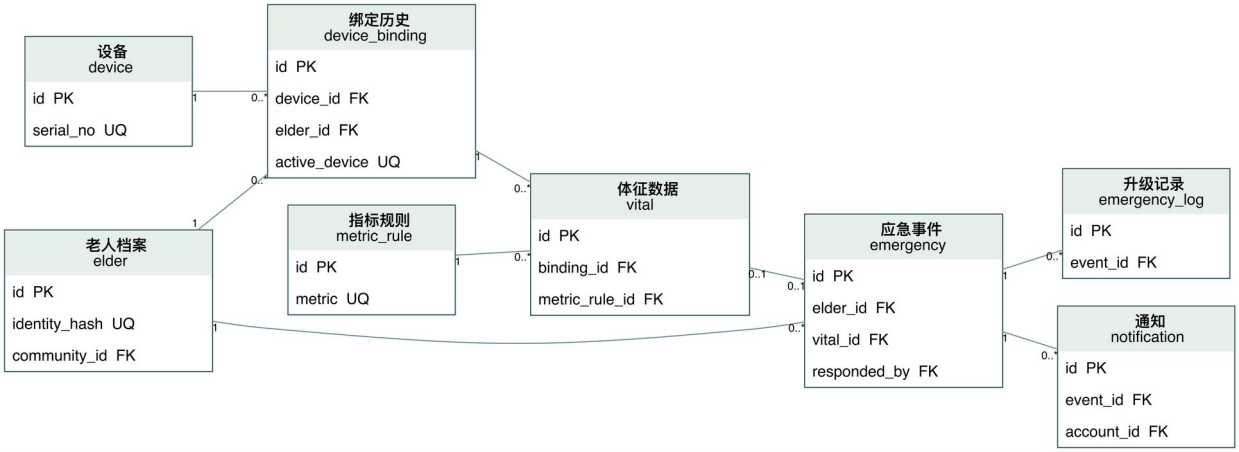
护工与排班局部模型



护工与排班局部 UML 类图

关联基数按业务含义和数据库外键标注。多个实体共用主数据时引用同一编号；可选关联使用允许空值的外键；一对多历史记录不通过覆盖旧行实现。

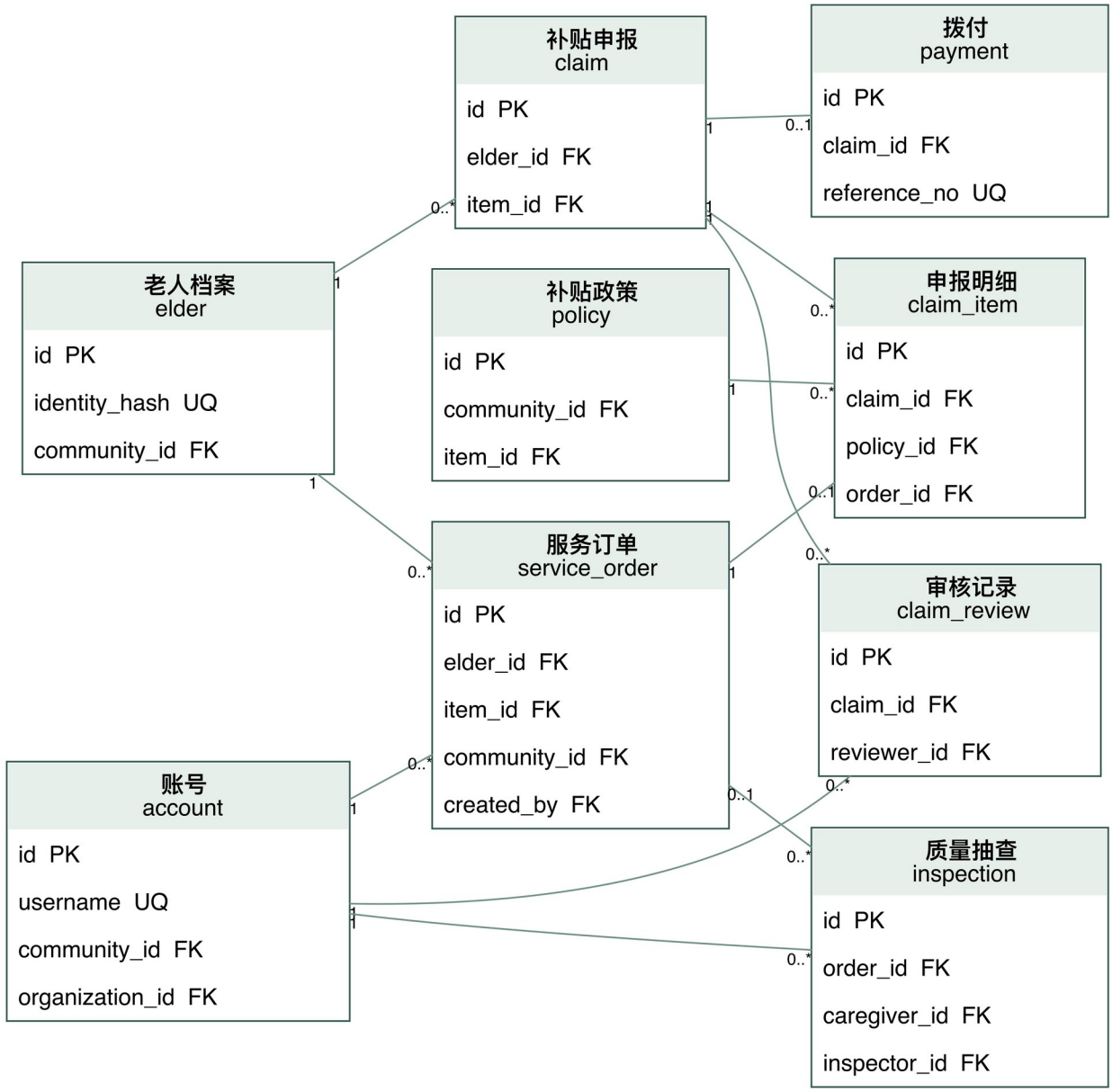
监测与应急局部模型



监测与应急局部 UML 类图

关联基数按业务含义和数据库外键标注。多个实体共用主数据时引用同一编号；可选关联使用允许空值的外键；一对多历史记录不通过覆盖旧行实现。

补贴与监管局部模型



补贴与监管局部 UML 类图

关联基数按业务含义和数据库外键标注。多个实体共用主数据时引用同一编号；可选关联使用允许空值的外键；一对多历史记录不通过覆盖旧行实现。

四、系统的逻辑结构设计

所有业务实体采用稳定的内部编号作为主键；身份号、设备序列号、证书号等业务唯一键另行约束。多值属性拆成关联表，实体联系通过外键表达。完整关系模式与所有字段见《数据字典与关系模式》。

范式判断方法

对每个关系先列出主键与其他候选键，再检查非主属性是否完全依赖候选键，并排除非键属性间的传递依赖。单列代理主键只能帮助标识记录，不能自动证明第三范式。

实体关系

community、organization、account、elder、assessment、service_item、care_bundle、caregiver、credential、shift、device、metric_rule、emergency、policy、inspection、complaint、audit_log 均以 id 确定本行事实。姓名不决定住址、机构或身份；评估得分与等级的对应口径随量表和版本而变化，本系统保存评估结论，不把 score 当作独立决定等级的全局函数。account 中 community_id 和 organization_id 表示授权范围，并非根据账号角色推导出的机构归属。

elder_tag 的候选键为 (elder_id,tag)；guardian 为 (elder_id,account_id)；bundle_item 为 (bundle_id,item_id)；caregiver_area 为 (caregiver_id,community_id)；plan_item 为 (plan_id,item_id)。各关联属性依赖完整组合键。外键指向对象的名称、地址等不重复放入关联表。

care_plan 只保存 assessment_id，老人归属从 assessment 派生；因此消除了 assessment_id → elder_id 在计划表内形成的传递依赖。v_plan_context 仅为查询展开归属，不新增事实。dispatch 只保存 shift_id，护工归属从 shift 派生，避免 shift_id → caregiver_id 的重复存储。v_dispatch_context 同理。

service_record 和 attendance 的 dispatch_id 是备用候选键；payment 的 claim_id 和 reference_no 均唯一；device 的 serial_no、credential 的 certificate_no、caregiver 的 account_id、elder 的 identity_hash、account 的 username 是业务候选键。

claim 的业务候选键是 (elder_id,item_id,period)。政策引用位于 claim_item 中，不在 claim 同时保存 policy_id 与可由它决定的 item_id。claim_item 的 order_id 唯一，确保同一服务订单不能重复获补；采用的政策、核定分钟和补贴单价属于该次核算事实。

历史事实与物理派生列

service_order 的地址、服务社区和金额是下单时的事实快照，并不由老人当前地址或目录当前指导价永久决定。claim_item 的 rate_snapshot 保存核算时价格，政策未来调整不能改变既有申报。device_binding 记录时间区间，vital 引用采集时的绑定，不由设备当前归属反推。

dispatch.active_order 和 device_binding.active_device 是物理生成列，用于条件唯一性约束；它们不属于独立的概念属性。shift.used_count、claim.amount 为受控物理汇总值；逻辑上可由明细推导，保留是为了容量并发控制和核算校验，必须通过事务同步或触发器核对。这些物理实现选择应与纯逻辑关系模式区别说明，不能把所有缓存都当作独立原始事实。

完整性与时间约束

主键、外键、UNIQUE 和 CHECK 约束负责基本合法性。计划和排班采用左闭右开区间，用 start1 < end2 且 end1 > start2 判定重叠。assert_plan 与 assert_shift 先锁定同一老人或护工再检查区间，应用事务使用 READ COMMITTED。派单先锁订单和护工，然后检查资质、区域、整改和预约冲突，并原子占用班次名额。事务失败回滚改派与名额变化。

历史禁止删除触发器保护评估、派单、服务记录、工时、体征、申报明细、审核、拨付和审计。已确认服务与已支付申报禁止普通修改。数据库管理员仍可管理结构与授权，本项目不声称能抵御拥有最高数据库权限的攻击者。

关系模式清单

社区：community(id [PK], name, district, area_code)。

服务机构：organization(id [PK], name, community_id)。

账号：account(id [PK], username, password_hash, role, community_id, organization_id, failures, locked_until, active)。

老人档案：elder(id [PK], name, identity_hash, identity_cipher, birth_date, address, community_id, status)。

老人标签：elder_tag(id [PK], elder_id, tag)。

家属关联：guardian(id [PK], elder_id, account_id, relation_name, phone_cipher)。

能力评估：assessment(id [PK], elder_id, scale_name, score, ability_level, assessor_id, assessed_at, valid_until)。

服务项目：service_item(id [PK], name, category, unit, price, duration_minutes, subsidizable)。

照护服务包：care_bundle(id [PK], name, ability_level)。

服务包明细：bundle_item(id [PK], bundle_id, item_id, weekly_frequency)。

护工：caregiver(id [PK], account_id, organization_id, name, status)。

项目资质：credential(id [PK], caregiver_id, item_id, certificate_no, certificate_level, valid_from, valid_until)。

护工服务区域：caregiver_area(id [PK], caregiver_id, community_id)。

照护计划：care_plan(id [PK], assessment_id, bundle_id, caregiver_id, starts_at, ends_at, status)。

计划明细：plan_item(id [PK], plan_id, item_id, weekly_frequency)。

排班：shift(id [PK], caregiver_id, starts_at, ends_at, capacity, used_count, status)。

服务订单：service_order(id [PK], elder_id, item_id, community_id, starts_at, ends_at, address, channel, amount, status, created_by, created_at)。

派单记录：dispatch(id [PK], order_id, shift_id, status, reason, assigned_at, accepted_at, active_order)。

服务与确认：service_record(id [PK], dispatch_id, notes, confirmed_by, confirmed_at, confirmation)。

工时记录：attendance(id [PK], dispatch_id, check_in, check_out, location_note, fence_ok, status, effective_minutes, reviewer_id, review_reason)。

设备：device(id [PK], serial_no, type_name, location_note, status)。

绑定历史：device_binding(id [PK], device_id, elder_id, bound_at, unbound_at, active_device)。

指标规则：metric_rule(id [PK], metric, low_value, high_value, response_seconds)。

体征数据：vital(id [PK], binding_id, metric_rule_id, sample_key, collected_at, metric_value, abnormal, source)。

应急事件：emergency(id [PK], elder_id, vital_id, source, level, occurred_at, deadline, status, responded_by, responded_at, arrived_at, result)。

升级记录：emergency_log(id [PK], event_id, level, created_at, message)。

通知：notification(id [PK], event_id, account_id, message, created_at)。

补贴政策：policy(id [PK], name, community_id, item_id, ability_level, rate, starts_at, ends_at)。

补贴申报：claim(id [PK], elder_id, item_id, period, amount, status)。

申报明细：claim_item(id [PK], claim_id, policy_id, order_id, amount, effective_minutes, rate_snapshot)。

审核记录：claim_review(id [PK], claim_id, reviewer_id, stage, decision, reason, created_at)。

拨付：payment(id [PK], claim_id, reference_no, amount, paid_at)。

质量抽查：inspection(id [PK], order_id, caregiver_id, inspector_id, method, score, issue, rectification, status, result)。

评价投诉：complaint(id [PK], order_id, account_id, rating, content, created_at)。

操作审计：audit_log(id [PK], table_name, row_id, operation, actor, old_status, new_status, created_at)。

所有外键及其目标列单列在 evidence/foreign-keys.tsv 与数据字典中；模式中的生成列仅服务于物理唯一性约束。

五、系统物理结构设计、持久层设计和关键业务代码

运行环境与分层

运行环境：Java 17.0.15，Spring Boot 3.4.4，Spring JDBC，MySQL 8.4.11，本机 macOS arm64。应用采用操作入口、业务事务、JDBC DAO、关系数据库四层组织。Windows 端提供 Java JAR 与本地 MySQL 初始化和启动脚本，不依赖 Docker；Windows 目标机运行仍需实测。



实际浏览器登录后的社区工作台

DAO 基础实现

```
package edu.bjfu.eldercare;

import org.springframework.jdbc.core.JdbcTemplate;

import org.springframework.jdbc.support.GeneratedKeyHolder;

import org.springframework.stereotype.Repository;

import java.sql.Statement;
```

```
import java.util.*;

@Repository

public class Store {

 public final JdbcTemplate jdbc;

 public Store(JdbcTemplate jdbc){this.jdbc=jdbc;}

 public List<Map<String,Object>> rows(String sql,Object...args){return
jdbc.queryForList(sql,args);}

 public Map<String,Object> one(String sql,Object...args){var
r=rows(sql,args);if(r.isEmpty())throw new Rule("记录不存在");return r.get(0);}

 public long insert(String sql,Object...args){var k=new
GeneratedKeyHolder();jdbc.update(c->{var
p=c.prepareStatement(sql,Statement.RETURN_GENERATED_KEYS);for(int
i=0;i<args.length;i++)p.setObject(i+1,args[i]);return p;},k);return
Objects.requireNonNull(k.getKey()).longValue();}

 public int update(String sql,Object...args){return jdbc.update(sql,args);}

 public long count(String sql,Object...args){return
Objects.requireNonNull(jdbc.queryForObject(sql,Long.class,args));}

 public void call(String name,Object...args){if(!
Set.of("assert_order","assign_order","verify_hours","submit_claim","expire_plans","escalate_
events").contains(name))throw new Rule("非法过程");jdbc.update("CALL
"+name+"("+String.join(",",Collections.nCopies(args.length,"?"))+")",args);}

 public static long id(Map<String,Object> m,String k){return
((Number)m.get(k)).longValue();}

}

class Rule extends RuntimeException {Rule(String m){super(m);}}
```

SQL 的用户值全部参数绑定。插入方法返回生成主键，查询和更新统一经过

JdbcTemplate。分页入口校验页码与大小并使用固定数据类别白名单。体征批量上传在同

一事务中逐条校验和写入，任一条越权或无效整体回滚。业务方法使用 @Transactional，过程不提前提交。

完整建表与索引语句

社区

```
CREATE TABLE community(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(80) NOT NULL,
  district VARCHAR(80) NOT NULL,
  area_code VARCHAR(30) NOT NULL UNIQUE);
```

服务机构

```
CREATE TABLE organization(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(100) NOT NULL,
  community_id BIGINT NOT NULL,
  FOREIGN KEY(community_id) REFERENCES community(id));
```

账号

```
CREATE TABLE account(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  username VARCHAR(50) NOT NULL UNIQUE,
  password_hash VARCHAR(100) NOT NULL,
  role ENUM('ADMIN','FAMILY','CAREGIVER','MANAGER','STREET','REGULATOR') NOT NULL,
  community_id BIGINT NOT NULL,
  organization_id BIGINT,
  failures INT NOT NULL DEFAULT 0,
  locked_until DATETIME,
  active BOOLEAN NOT NULL DEFAULT TRUE,
  FOREIGN KEY(community_id) REFERENCES community(id),
```

```
  FOREIGN KEY(organization_id) REFERENCES organization(id));
```

老人档案

```
CREATE TABLE elder(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(60) NOT NULL,
  identity_hash CHAR(64) NOT NULL UNIQUE,
  identity_cipher TEXT NOT NULL,
  birth_date DATE NOT NULL,
  address VARCHAR(200) NOT NULL,
  community_id BIGINT NOT NULL,
  status ENUM('ACTIVE','INACTIVE') NOT NULL DEFAULT 'ACTIVE',
  FOREIGN KEY(community_id) REFERENCES community(id));
```

老人标签

```
CREATE TABLE elder_tag(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  elder_id BIGINT NOT NULL,
  tag ENUM('ALONE','DISABLED','LOW_INCOME') NOT NULL,
  UNIQUE(elder_id,tag),
  FOREIGN KEY(elder_id) REFERENCES elder(id));
```

家属关联

```
CREATE TABLE guardian(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  elder_id BIGINT NOT NULL,
  account_id BIGINT NOT NULL,
  relation_name VARCHAR(30) NOT NULL,
  phone_cipher TEXT NOT NULL,
  UNIQUE(elder_id,account_id),
  FOREIGN KEY(elder_id) REFERENCES elder(id),
  FOREIGN KEY(account_id) REFERENCES account(id));
```

能力评估

```
CREATE TABLE assessment(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  elder_id BIGINT NOT NULL,
  scale_name VARCHAR(50) NOT NULL,
  score INT NOT NULL,
  ability_level ENUM('SELF','PARTIAL','SEVERE') NOT NULL,
  assessor_id BIGINT NOT NULL,
  assessed_at DATETIME NOT NULL,
  valid_until DATETIME NOT NULL,
  CHECK(score BETWEEN 0 AND 100),
  CHECK(valid_until>assessed_at),
  FOREIGN KEY(elder_id) REFERENCES elder(id),
  FOREIGN KEY(assessor_id) REFERENCES account(id),
  INDEX idx_assessment_elder(elder_id,valid_until));
```

服务项目

```
CREATE TABLE service_item(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(80) NOT NULL,
  category VARCHAR(30) NOT NULL,
  unit ENUM('VISIT','HOUR') NOT NULL,
  price DECIMAL(10,2) NOT NULL,
  duration_minutes INT NOT NULL,
  subsidizable BOOLEAN NOT NULL,
  CHECK(price>=0),
  CHECK(duration_minutes>0));
```

照护服务包

```
CREATE TABLE care_bundle(id BIGINT PRIMARY KEY AUTO_INCREMENT,
```

```
  name VARCHAR(80) NOT NULL,
  ability_level ENUM('SELF','PARTIAL','SEVERE') NOT NULL);
```

服务包明细

```
CREATE TABLE bundle_item(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  bundle_id BIGINT NOT NULL,
  item_id BIGINT NOT NULL,
  weekly_frequency INT NOT NULL,
  CHECK(weekly_frequency>0),
  UNIQUE(bundle_id,item_id),
  FOREIGN KEY(bundle_id) REFERENCES care_bundle(id),
  FOREIGN KEY(item_id) REFERENCES service_item(id));
```

护工

```
CREATE TABLE caregiver(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  account_id BIGINT NOT NULL UNIQUE,
  organization_id BIGINT NOT NULL,
  name VARCHAR(60) NOT NULL,
  status ENUM('ACTIVE','LEAVE','LEFT') NOT NULL DEFAULT 'ACTIVE',
  FOREIGN KEY(account_id) REFERENCES account(id),
  FOREIGN KEY(organization_id) REFERENCES organization(id));
```

项目资质

```
CREATE TABLE credential(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  caregiver_id BIGINT NOT NULL,
  item_id BIGINT NOT NULL,
  certificate_no VARCHAR(80) NOT NULL UNIQUE,
  certificate_level VARCHAR(30) NOT NULL,
  valid_from DATETIME NOT NULL,
```

```
  valid_until DATETIME NOT NULL,
  CHECK(valid_until>valid_from),
  FOREIGN KEY(caregiver_id) REFERENCES caregiver(id),
  FOREIGN KEY(item_id) REFERENCES service_item(id));
```

护工服务区域

```
CREATE TABLE caregiver_area(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  caregiver_id BIGINT NOT NULL,
  community_id BIGINT NOT NULL,
  UNIQUE(caregiver_id,community_id),
  FOREIGN KEY(caregiver_id) REFERENCES caregiver(id),
  FOREIGN KEY(community_id) REFERENCES community(id));
```

照护计划

```
CREATE TABLE care_plan(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  assessment_id BIGINT NOT NULL,
  bundle_id BIGINT NOT NULL,
  caregiver_id BIGINT NOT NULL,
  starts_at DATETIME NOT NULL,
  ends_at DATETIME NOT NULL,
  status ENUM('ACTIVE','PAUSED','EXPIRED','ENDED') NOT NULL DEFAULT 'ACTIVE',
  CHECK(ends_at>starts_at),
  FOREIGN KEY(assessment_id) REFERENCES assessment(id),
  FOREIGN KEY(bundle_id) REFERENCES care_bundle(id),
  FOREIGN KEY(caregiver_id) REFERENCES caregiver(id),
  INDEX idx_plan_assessment(assessment_id,status,starts_at,ends_at));
```

计划明细

```
CREATE TABLE plan_item(id BIGINT PRIMARY KEY AUTO_INCREMENT,
```

```
  plan_id BIGINT NOT NULL,
  item_id BIGINT NOT NULL,
  weekly_frequency INT NOT NULL,
  CHECK(weekly_frequency>0),
  UNIQUE(plan_id,item_id),
  FOREIGN KEY(plan_id) REFERENCES care_plan(id),
  FOREIGN KEY(item_id) REFERENCES service_item(id));
```

排班

```
CREATE TABLE shift(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  caregiver_id BIGINT NOT NULL,
  starts_at DATETIME NOT NULL,
  ends_at DATETIME NOT NULL,
  capacity INT NOT NULL,
  used_count INT NOT NULL DEFAULT 0,
  status ENUM('OPEN','CLOSED') NOT NULL DEFAULT 'OPEN',
  CHECK(ends_at>starts_at),
  CHECK(capacity>0 AND used_count>=0 AND used_count<=capacity),
  FOREIGN KEY(caregiver_id) REFERENCES caregiver(id),
  INDEX idx_shift_caregiver(caregiver_id,starts_at,ends_at));
```

服务订单

```
CREATE TABLE service_order(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  elder_id BIGINT NOT NULL,
  item_id BIGINT NOT NULL,
  community_id BIGINT NOT NULL,
  starts_at DATETIME NOT NULL,
  ends_at DATETIME NOT NULL,
```

```
  address VARCHAR(200) NOT NULL,
  channel ENUM('PHONE','FAMILY','COMMUNITY') NOT NULL,
  amount DECIMAL(10,2) NOT NULL,
  status
ENUM('PENDING','ASSIGNED','ACCEPTED','SERVING','CONFIRM','COMPLETED','CANCELLED') NOT NULL DEFAULT 'PENDING',
  created_by BIGINT NOT NULL,
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CHECK(ends_at>starts_at),
  CHECK(amount>=0),
  FOREIGN KEY(elder_id) REFERENCES elder(id),
  FOREIGN KEY(item_id) REFERENCES service_item(id),
  FOREIGN KEY(community_id) REFERENCES community(id),
  FOREIGN KEY(created_by) REFERENCES account(id),
  INDEX idx_order_status_time(status,starts_at),
  INDEX idx_order_elder(elder_id,starts_at));
```

派单记录

```
CREATE TABLE dispatch(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  order_id BIGINT NOT NULL,
  shift_id BIGINT NOT NULL,
  status ENUM('WAITING','ACCEPTED','FINISHED','REASSIGNED','REJECTED') NOT NULL DEFAULT 'WAITING',
  reason VARCHAR(200) NOT NULL DEFAULT '首次派单',
  assigned_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  accepted_at DATETIME,
  active_order BIGINT GENERATED ALWAYS AS (CASE WHEN status IN ('WAITING','ACCEPTED') THEN order_id ELSE NULL END) STORED,
```

```
  UNIQUE(active_order),
  FOREIGN KEY(order_id) REFERENCES service_order(id),
  FOREIGN KEY(shift_id) REFERENCES shift(id),
  INDEX idx_dispatch_shift(shift_id,status));
```

服务与确认

```
CREATE TABLE service_record(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  dispatch_id BIGINT NOT NULL UNIQUE,
  notes VARCHAR(1000) NOT NULL,
  confirmed_by BIGINT,
  confirmed_at DATETIME,
  confirmation VARCHAR(300),
  FOREIGN KEY(dispatch_id) REFERENCES dispatch(id),
  FOREIGN KEY(confirmed_by) REFERENCES account(id));
```

工时记录

```
CREATE TABLE attendance(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  dispatch_id BIGINT NOT NULL UNIQUE,
  check_in DATETIME NOT NULL,
  check_out DATETIME,
  location_note VARCHAR(200) NOT NULL,
  fence_ok BOOLEAN NOT NULL,
  status ENUM('PENDING','REVIEW','VALID','INVALID') NOT NULL DEFAULT 'PENDING',
  effective_minutes INT NOT NULL DEFAULT 0,
  reviewer_id BIGINT,
  review_reason VARCHAR(300),
  CHECK(check_out IS NULL OR check_out>check_in),
  CHECK(effective_minutes>=0),
```

```
  FOREIGN KEY(dispatch_id) REFERENCES dispatch(id),
  FOREIGN KEY(reviewer_id) REFERENCES account(id));
```

设备

```
CREATE TABLE device(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  serial_no VARCHAR(80) NOT NULL UNIQUE,
  type_name VARCHAR(50) NOT NULL,
  location_note VARCHAR(200) NOT NULL,
  status ENUM('ACTIVE','OFFLINE','RETIRED') NOT NULL DEFAULT 'ACTIVE');
```

绑定历史

```
CREATE TABLE device_binding(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  device_id BIGINT NOT NULL,
  elder_id BIGINT NOT NULL,
  bound_at DATETIME NOT NULL,
  unbound_at DATETIME,
  active_device BIGINT GENERATED ALWAYS AS (CASE WHEN unbound_at IS NULL
THEN device_id ELSE NULL END) STORED,
  UNIQUE(active_device),
  CHECK(unbound_at IS NULL OR unbound_at>bound_at),
  FOREIGN KEY(device_id) REFERENCES device(id),
  FOREIGN KEY(elder_id) REFERENCES elder(id));
```

指标规则

```
CREATE TABLE metric_rule(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  metric VARCHAR(60) NOT NULL UNIQUE,
  low_value DECIMAL(12,3) NOT NULL,
  high_value DECIMAL(12,3) NOT NULL,
  response_seconds INT NOT NULL,
```

```
  CHECK(high_value>low_value),
  CHECK(response_seconds>0));
```

体征数据

```
CREATE TABLE vital(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  binding_id BIGINT NOT NULL,
  metric_rule_id BIGINT NOT NULL,
  sample_key VARCHAR(80) NOT NULL,
  collected_at DATETIME NOT NULL,
  metric_value DECIMAL(12,3) NOT NULL,
  abnormal BOOLEAN NOT NULL DEFAULT FALSE,
  source VARCHAR(50) NOT NULL,
  UNIQUE(binding_id,sample_key),
  FOREIGN KEY(binding_id) REFERENCES device_binding(id),
  FOREIGN KEY(metric_rule_id) REFERENCES metric_rule(id));
```

应急事件

```
CREATE TABLE emergency(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  elder_id BIGINT NOT NULL,
  vital_id BIGINT UNIQUE,
  source VARCHAR(50) NOT NULL,
  level INT NOT NULL DEFAULT 1,
  occurred_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  deadline DATETIME NOT NULL,
  status ENUM('UNCONFIRMED','OPEN','RESPONDED','CLOSED','FALSE_ALARM') NOT
NULL,
  responded_by BIGINT,
  responded_at DATETIME,
```

```
  arrived_at DATETIME,
  result VARCHAR(1000),
  CHECK(level BETWEEN 1 AND 3),
  FOREIGN KEY(elder_id) REFERENCES elder(id),
  FOREIGN KEY(vital_id) REFERENCES vital(id),
  FOREIGN KEY(responded_by) REFERENCES account(id),
  INDEX idx_emergency_deadline(status,deadline));
```

升级记录

```
CREATE TABLE emergency_log(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  event_id BIGINT NOT NULL,
  level INT NOT NULL,
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  message VARCHAR(300) NOT NULL,
  UNIQUE(event_id,level),
  FOREIGN KEY(event_id) REFERENCES emergency(id));
```

通知

```
CREATE TABLE notification(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  event_id BIGINT NOT NULL,
  account_id BIGINT NOT NULL,
  message VARCHAR(300) NOT NULL,
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  UNIQUE(event_id,account_id,message),
  FOREIGN KEY(event_id) REFERENCES emergency(id),
  FOREIGN KEY(account_id) REFERENCES account(id));
```

补贴政策

```
CREATE TABLE policy(id BIGINT PRIMARY KEY AUTO_INCREMENT,
```

```
  name VARCHAR(100) NOT NULL,
  community_id BIGINT NOT NULL,
  item_id BIGINT NOT NULL,
  ability_level ENUM('SELF','PARTIAL','SEVERE') NOT NULL,
  rate DECIMAL(10,2) NOT NULL,
  starts_at DATETIME NOT NULL,
  ends_at DATETIME NOT NULL,
  CHECK(rate>=0),
  CHECK(ends_at>starts_at),
  FOREIGN KEY(community_id) REFERENCES community(id),
  FOREIGN KEY(item_id) REFERENCES service_item(id));
```

补贴申报

```
CREATE TABLE claim(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  elder_id BIGINT NOT NULL,
  item_id BIGINT NOT NULL,
  period CHAR(7) NOT NULL,
  amount DECIMAL(12,2) NOT NULL DEFAULT 0,
  status
ENUM('DRAFT','SUBMITTED','STREET_APPROVED','APPROVED','REJECTED','PAID')
NOT NULL DEFAULT 'DRAFT',
  UNIQUE(elder_id,item_id,period),
  CHECK(amount>=0),
  FOREIGN KEY(elder_id) REFERENCES elder(id),
  FOREIGN KEY(item_id) REFERENCES service_item(id));
```

申报明细

```
CREATE TABLE claim_item(id BIGINT PRIMARY KEY AUTO_INCREMENT,
```

```
  claim_id BIGINT NOT NULL,
  policy_id BIGINT NOT NULL,
  order_id BIGINT NOT NULL UNIQUE,
  amount DECIMAL(12,2) NOT NULL,
  effective_minutes INT NOT NULL,
  rate_snapshot DECIMAL(10,2) NOT NULL,
  CHECK(amount>=0 AND effective_minutes>0),
  FOREIGN KEY(policy_id) REFERENCES policy(id),
  FOREIGN KEY(claim_id) REFERENCES claim(id),
  FOREIGN KEY(order_id) REFERENCES service_order(id));
```

审核记录

```
CREATE TABLE claim_review(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  claim_id BIGINT NOT NULL,
  reviewer_id BIGINT NOT NULL,
  stage ENUM('STREET','REGULATOR') NOT NULL,
  decision ENUM('APPROVE','REJECT') NOT NULL,
  reason VARCHAR(300) NOT NULL,
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY(claim_id) REFERENCES claim(id),
  FOREIGN KEY(reviewer_id) REFERENCES account(id));
```

拨付

```
CREATE TABLE payment(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  claim_id BIGINT NOT NULL UNIQUE,
  reference_no VARCHAR(80) NOT NULL UNIQUE,
  amount DECIMAL(12,2) NOT NULL,
  paid_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
```

```
  CHECK(amount>0),
  FOREIGN KEY(claim_id) REFERENCES claim(id));
```

质量抽查

```
CREATE TABLE inspection(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  order_id BIGINT,
  caregiver_id BIGINT,
  inspector_id BIGINT NOT NULL,
  method ENUM('PHONE','VISIT') NOT NULL,
  score INT NOT NULL,
  issue VARCHAR(300) NOT NULL,
  rectification VARCHAR(300) NOT NULL,
  status ENUM('OPEN','CLOSED') NOT NULL DEFAULT 'OPEN',
  result VARCHAR(300),
  CHECK((order_id IS NULL)<>(caregiver_id IS NULL)),
  CHECK(score BETWEEN 0 AND 100),
  FOREIGN KEY(order_id) REFERENCES service_order(id),
  FOREIGN KEY(caregiver_id) REFERENCES caregiver(id),
  FOREIGN KEY(inspector_id) REFERENCES account(id));
```

评价投诉

```
CREATE TABLE complaint(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  order_id BIGINT NOT NULL,
  account_id BIGINT NOT NULL,
  rating INT NOT NULL,
  content VARCHAR(500) NOT NULL,
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CHECK(rating BETWEEN 1 AND 5),
```

```
  FOREIGN KEY(order_id) REFERENCES service_order(id),
  FOREIGN KEY(account_id) REFERENCES account(id));
```

操作审计

```
CREATE TABLE audit_log(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  table_name VARCHAR(50) NOT NULL,
  row_id BIGINT NOT NULL,
  operation VARCHAR(20) NOT NULL,
  actor VARCHAR(100) NOT NULL,
  old_status VARCHAR(100),
  new_status VARCHAR(100),
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  INDEX idx_audit_object(table_name,row_id));
```

业务视图与内部关联视图

五条业务线各提供三个不同用途的视图，共 15 个业务视图。另有 v_plan_context 和 v_dispatch_context 两个内部视图，派生老人、护工归属，避免基础表重复保存。视图提供查询结构，具体人员范围仍由业务授权过滤决定。

```
CREATE VIEW v_assessment_due AS SELECT e.id,e.name,e.community_id,a.id assessment_id,a.valid_until FROM elder e JOIN assessment a ON a.elder_id=e.id WHERE a.valid_until BETWEEN NOW() AND NOW()+INTERVAL 30 DAY;
CREATE VIEW v_active_plan AS SELECT p.id,p.elder_id,e.name,e.community_id,p.starts_at,p.ends_at,c.name caregiver_name FROM v_plan_context p JOIN elder e ON e.id=p.elder_id JOIN caregiver c ON c.id=p.caregiver_id WHERE p.status='ACTIVE' AND NOW()>=p.starts_at AND NOW()<p.ends_at;
CREATE VIEW v_family_care AS SELECT g.account_id,e.id elder_id,e.name,e.community_id,p.id plan_id,p.status FROM guardian g JOIN elder e ON e.id=g.elder_id LEFT JOIN v_plan_context p ON p.elder_id=e.id AND p.status='ACTIVE';
```

```
CREATE VIEW v_dispatch_overdue AS SELECT
o.id,o.elder_id,o.community_id,o.starts_at,o.status FROM service_order o WHERE
o.status='PENDING' AND o.created_at<NOW()-INTERVAL 15 MINUTE;

CREATE VIEW v_wait_confirmation AS SELECT o.id,o.elder_id,o.community_id,r.id
record_id FROM service_order o JOIN v_dispatch_context d ON d.order_id=o.id JOIN
service_record r ON r.dispatch_id=d.id WHERE o.status='CONFIRM' AND r.confirmed_at
IS NULL;

CREATE VIEW v_reassign_history AS SELECT
d.id,d.order_id,d.caregiver_id,c.name,d.reason,d.assigned_at,o.community_id FROM
v_dispatch_context d JOIN caregiver c ON c.id=d.caregiver_id JOIN service_order o ON
o.id=d.order_id WHERE d.status='REASSIGNED';

CREATE VIEW v_credential_due AS SELECT
x.id,c.name,c.organization_id,x.item_id,x.valid_until FROM credential x JOIN caregiver c ON
c.id=x.caregiver_id WHERE x.valid_until<NOW()+INTERVAL 30 DAY;

CREATE VIEW v_shift_capacity AS SELECT
s.id,s.caregiver_id,c.organization_id,s.starts_at,s.ends_at,s.capacity-s.used_count available
FROM shift s JOIN caregiver c ON c.id=s.caregiver_id WHERE s.status='OPEN';

CREATE VIEW v_hours_review AS SELECT
a.id,d.caregiver_id,c.organization_id,d.order_id,a.check_in,a.check_out,a.location_note,a.sta
tus FROM attendance a JOIN v_dispatch_context d ON d.id=a.dispatch_id JOIN caregiver
c ON c.id=d.caregiver_id WHERE a.status='REVIEW';

CREATE VIEW v_device_current AS SELECT
d.id,d.serial_no,d.status,b.elder_id,e.community_id,b.bound_at FROM device d LEFT JOIN
device_binding b ON b.device_id=d.id AND b.unbound_at IS NULL LEFT JOIN elder e ON
e.id=b.elder_id;

CREATE VIEW v_abnormal_vital AS SELECT
v.id,b.elder_id,e.community_id,m.metric,v.metric_value,v.collected_at FROM vital v JOIN
device_binding b ON b.id=v.binding_id JOIN elder e ON e.id=b.elder_id JOIN metric_rule m
ON m.id=v.metric_rule_id WHERE v.abnormal=1;

CREATE VIEW v_emergency_pending AS SELECT
x.id,x.elder_id,e.community_id,x.status,x.level,x.deadline,TIMESTAMPDIFF(SECOND,NOW()
```

```
,x.deadline) remaining_seconds FROM emergency x JOIN elder e ON e.id=x.elder_id
WHERE x.status NOT IN('CLOSED','FALSE_ALARM');

CREATE VIEW v_claim_eligible AS SELECT
o.id,o.elder_id,o.item_id,o.community_id,o.amount,a.effective_minutes FROM service_order
o JOIN v_dispatch_context d ON d.order_id=o.id JOIN attendance a ON a.dispatch_id=d.id
JOIN service_record r ON r.dispatch_id=d.id JOIN service_item s ON s.id=o.item_id
WHERE o.status='COMPLETED' AND d.status='FINISHED' AND r.confirmed_at IS NOT
NULL AND a.status='VALID' AND s.subsidizable=1 AND NOT EXISTS(SELECT 1 FROM
claim_item ci WHERE ci.order_id=o.id);

CREATE VIEW v_claim_progress AS SELECT
c.id,c.elder_id,e.community_id,c.period,c.amount,c.status,p.reference_no,p.paid_at FROM
claim c JOIN elder e ON e.id=c.elder_id LEFT JOIN payment p ON p.claim_id=c.id;

CREATE VIEW v_rectification AS SELECT
i.id,i.order_id,i.caregiver_id,i.issue,i.rectification,a.community_id FROM inspection i JOIN
account a ON a.id=i.inspector_id WHERE i.status='OPEN';

CREATE VIEW v_plan_context AS SELECT p.*,a.elder_id FROM care_plan p JOIN
assessment a ON a.id=p.assessment_id;

CREATE VIEW v_dispatch_context AS SELECT d.*,s.caregiver_id FROM dispatch d JOIN
shift s ON s.id=d.shift_id;
```

各业务 SQL 的两种实现

A1 有效评估与计划的重度失能老人

连接去重与 EXISTS 半连接比较，避免一人多记录重复输出。

实现 A

```
SELECT DISTINCT e.id,e.name FROM elder e JOIN assessment a ON a.elder_id=e.id JOIN
v_plan_context p ON p.assessment_id=a.id WHERE a.ability_level='SEVERE' AND
p.status='ACTIVE' AND a.valid_until>NOW();
```

实现 B

```
SELECT e.id,e.name FROM elder e WHERE EXISTS(SELECT 1 FROM assessment a JOIN v_plan_context p ON p.assessment_id=a.id WHERE a.elder_id=e.id AND a.ability_level='SEVERE' AND p.status='ACTIVE' AND a.valid_until>NOW());
```

两种实现已对当前测试数据执行并逐行比较，结果一致。执行计划和实际结果见 evidence/query-comparison.json。

A2 每个老人的评估次数

左连接聚合与相关子查询；保留零次评估老人。

实现 A

```
SELECT e.id,COUNT(a.id) n FROM elder e LEFT JOIN assessment a ON a.elder_id=e.id GROUP BY e.id;
```

实现 B

```
SELECT e.id,(SELECT COUNT(*) FROM assessment a WHERE a.elder_id=e.id) n FROM elder e;
```

两种实现已对当前测试数据执行并逐行比较，结果一致。执行计划和实际结果见 evidence/query-comparison.json。

A3 没有生效计划的老人

反连接与 NOT EXISTS，不使用含 NULL 风险的 NOT IN。

实现 A

```
SELECT e.id FROM elder e LEFT JOIN v_plan_context p ON p.elder_id=e.id AND p.status='ACTIVE' WHERE p.id IS NULL;
```

实现 B

```
SELECT e.id FROM elder e WHERE NOT EXISTS(SELECT 1 FROM v_plan_context p WHERE p.elder_id=e.id AND p.status='ACTIVE');
```

两种实现已对当前测试数据执行并逐行比较，结果一致。执行计划和实际结果见 evidence/query-comparison.json。

B1 有改派历史的订单

多表连接去重与 EXISTS，结果粒度为订单。

实现 A

```
SELECT DISTINCT o.id,e.name FROM service_order o JOIN elder e ON e.id=o.elder_id JOIN v_dispatch_context d ON d.order_id=o.id WHERE d.status='REASSIGNED';
```

实现 B

```
SELECT o.id,e.name FROM service_order o JOIN elder e ON e.id=o.elder_id WHERE EXISTS(SELECT 1 FROM v_dispatch_context d WHERE d.order_id=o.id AND d.status='REASSIGNED');
```

两种实现已对当前测试数据执行并逐行比较，结果一致。执行计划和实际结果见 evidence/query-comparison.json。

B2 各社区订单金额

聚合与相关子查询，空集合显式转零。

实现 A

```
SELECT c.id,COALESCE(SUM(o.amount),0) total FROM community c LEFT JOIN service_order o ON o.community_id=c.id GROUP BY c.id;
```

实现 B

```
SELECT c.id,COALESCE((SELECT SUM(o.amount) FROM service_order o WHERE o.community_id=c.id),0) total FROM community c;
```

两种实现已对当前测试数据执行并逐行比较，结果一致。执行计划和实际结果见 evidence/query-comparison.json。

B3 待确认订单

保留订单粒度，防止多次派单重复统计。

实现 A

```
SELECT DISTINCT o.id FROM service_order o JOIN v_dispatch_context d ON d.order_id=o.id JOIN service_record r ON r.dispatch_id=d.id WHERE o.status='CONFIRM' AND r.confirmed_at IS NULL;
```

实现 B

```
SELECT o.id FROM service_order o WHERE o.status='CONFIRM' AND EXISTS(SELECT 1 FROM v_dispatch_context d JOIN service_record r ON r.dispatch_id=d.id WHERE d.order_id=o.id AND r.confirmed_at IS NULL);
```

两种实现已对当前测试数据执行并逐行比较，结果一致。执行计划和实际结果见 evidence/query-comparison.json。

C1 护工有效工时

三表聚合与子查询。未服务护工仍返回零。

实现 A

```
SELECT c.id,COALESCE(SUM(a.effective_minutes),0) minutes FROM caregiver c LEFT JOIN v_dispatch_context d ON d.caregiver_id=c.id LEFT JOIN attendance a ON a.dispatch_id=d.id AND a.status='VALID' GROUP BY c.id;
```

实现 B

```
SELECT c.id,COALESCE((SELECT SUM(a.effective_minutes) FROM v_dispatch_context d JOIN attendance a ON a.dispatch_id=d.id WHERE d.caregiver_id=c.id AND a.status='VALID'),0) minutes FROM caregiver c;
```

两种实现已对当前测试数据执行并逐行比较，结果一致。执行计划和实际结果见 evidence/query-comparison.json。

C2 有空余名额的护工

该查询只检查容量，不等同于完整派单资格推荐。

实现 A

```
SELECT DISTINCT c.id FROM caregiver c JOIN shift s ON s.caregiver_id=c.id WHERE s.status='OPEN' AND s.used_count<s.capacity;
```

实现 B

```
SELECT c.id FROM caregiver c WHERE EXISTS(SELECT 1 FROM shift s WHERE
s.caregiver_id=c.id AND s.status='OPEN' AND s.used_count<s.capacity);
```

两种实现已对当前测试数据执行并逐行比较，结果一致。执行计划和实际结果见 evidence/query-comparison.json。

C3 证书到期护工

到期清单与完整资质判断分离，一张证书到期不代表所有项目失效。

实现 A

```
SELECT DISTINCT c.id FROM caregiver c JOIN credential x ON x.caregiver_id=c.id
WHERE x.valid_until<NOW();
```

实现 B

```
SELECT c.id FROM caregiver c WHERE EXISTS(SELECT 1 FROM credential x WHERE
x.caregiver_id=c.id AND x.valid_until<NOW());
```

两种实现已对当前测试数据执行并逐行比较，结果一致。执行计划和实际结果见 evidence/query-comparison.json。

D1 老人异常体征数量

绑定历史关联，不能仅通过设备当前归属统计。

实现 A

```
SELECT e.id,COUNT(v.id) n FROM elder e LEFT JOIN device_binding b ON b.elder_id=e.id
LEFT JOIN vital v ON v.binding_id=b.id AND v.abnormal=1 GROUP BY e.id;
```

实现 B

```
SELECT e.id,(SELECT COUNT(*) FROM device_binding b JOIN vital v ON v.binding_id=b.id
WHERE b.elder_id=e.id AND v.abnormal=1) n FROM elder e;
```

两种实现已对当前测试数据执行并逐行比较，结果一致。执行计划和实际结果见 evidence/query-comparison.json。

D2 存在未关闭事件的老人

连接去重与 EXISTS，待确认事件仍纳入。

实现 A

```
SELECT DISTINCT e.id FROM elder e JOIN emergency x ON x.elder_id=e.id WHERE x.status NOT IN('CLOSED','FALSE_ALARM');
```

实现 B

```
SELECT e.id FROM elder e WHERE EXISTS(SELECT 1 FROM emergency x WHERE x.elder_id=e.id AND x.status NOT IN('CLOSED','FALSE_ALARM'));
```

两种实现已对当前测试数据执行并逐行比较，结果一致。执行计划和实际结果见 evidence/query-comparison.json。

D3 当前未绑定设备

历史绑定不应排除当前空闲设备。

实现 A

```
SELECT d.id FROM device d LEFT JOIN device_binding b ON b.device_id=d.id AND b.unbound_at IS NULL WHERE b.id IS NULL;
```

实现 B

```
SELECT d.id FROM device d WHERE NOT EXISTS(SELECT 1 FROM device_binding b WHERE b.device_id=d.id AND b.unbound_at IS NULL);
```

两种实现已对当前测试数据执行并逐行比较，结果一致。执行计划和实际结果见 evidence/query-comparison.json。

E1 按社区汇总已申报金额

当前管理辖区统计口径；历史财政归属应改按政策辖区统计。

实现 A

SELECT co.id,COALESCE(SUM(c.amount),0) amount FROM community co LEFT JOIN elder e ON e.community_id=co.id LEFT JOIN claim c ON c.elder_id=e.id AND c.status<>'DRAFT' GROUP BY co.id;

实现 B

SELECT co.id,COALESCE((SELECT SUM(c.amount) FROM elder e JOIN claim c ON c.elder_id=e.id WHERE e.community_id=co.id AND c.status<>'DRAFT'),0) amount FROM community co;

两种实现已对当前测试数据执行并逐行比较，结果一致。执行计划和实际结果见 evidence/query-comparison.json。

E2 待拨付申报

反连接与 NOT EXISTS。

实现 A

SELECT c.id FROM claim c LEFT JOIN payment p ON p.claim_id=c.id WHERE c.status='APPROVED' AND p.id IS NULL;

实现 B

SELECT c.id FROM claim c WHERE c.status='APPROVED' AND NOT EXISTS(SELECT 1 FROM payment p WHERE p.claim_id=c.id);

两种实现已对当前测试数据执行并逐行比较，结果一致。执行计划和实际结果见 evidence/query-comparison.json。

E3 各申报明细合计

汇总后与主表金额比较，用于核算一致性检查。

实现 A

SELECT c.id,COALESCE(SUM(i.amount),0) amount FROM claim c LEFT JOIN claim_item i ON i.claim_id=c.id GROUP BY c.id;

实现 B

```
SELECT c.id,COALESCE((SELECT SUM(i.amount) FROM claim_item i WHERE
i.claim_id=c.id),0) amount FROM claim c;
```

两种实现已对当前测试数据执行并逐行比较，结果一致。执行计划和实际结果见 evidence/query-comparison.json。

六、项目运维管理和优化

环境和边界

Java 17、Spring Boot 3.4.4、Spring JDBC，真实 MySQL 8.4.11。本机独立实例绑定 127.0.0.1:33316，应用绑定 127.0.0.1:18080。Windows 使用安装好的 Java 和 MySQL，通过配置更换 JDBC 地址，Docker 不参与运行。Windows 脚本尚需在目标机器执行验证。

设备上传和地理围栏为课程模拟。指标上下限、300 秒初始响应时限、逐次 5 分钟升级及每次 30 元补贴均为演示配置，不能用于真实医疗或财政业务。补贴按服务月份归集，当前一个老人同项目同月只能有一份申报，驳回后重报尚未提供正式版本流程。

过程与触发器对应

过程由 Java 事务调用，过程内部不 COMMIT；一个业务方法失败时统一回滚，不能在调用链中提前提交。定时任务每 30 秒检查到期和超时，属于应用进程内调度，进程停止后暂停，恢复后补处理。重要入口还会即时校验有效期，不能只依赖定时状态更新。

索引方案

1. idx_assessment_elder(elder_id,valid_until)：按老人查有效评估，适用于资格判断和复评清单。纯到期全库扫描可能仍需时间列开头的索引，应根据实际规模再决定。

2. idx_plan_assessment(assessment_id,status,starts_at,ends_at)：关联评估并筛选计划。重叠判断仍需比较结束时刻，不能把索引等同于区间约束。

3. idx_order_status_time(status,starts_at)：调度状态与预约区间查询。证据中对比忽略该索引与使用该索引的 EXPLAIN；小数据集不据此声称显著提速。

4. idx_order_elder(elder_id,starts_at)：老人订单分页、历史服务和月度申报筛选。

5. idx_shift_caregiver(caregiver_id,starts_at,ends_at)：查护工时段及候选班次。

6. idx_emergency_deadline(status,deadline)：定时筛选未响应且超时的应急事件。

7. idx_audit_object(table_name,row_id)：追溯指定业务记录的状态历史。

多对多表使用组合 UNIQUE 防止重复关系。生成唯一列 active_order 和 active_device 分别保证一单只有一个有效派单、一设备只有一个当前绑定。外键自动或显式建立必要访问索引，避免对每个字段机械加索引。

登录和敏感字段

密码以 BCrypt 12 轮加盐摘要保存。连续失败 5 次后锁定 15 分钟，失败计数使用独立数据库事务提交，避免抛出登录错误时把计数一起回滚。会话由服务端保存随机令牌，固定 30 分钟到期，退出立即删除，应用重启使既有会话失效。当前是单实例课程实现，未做分布式会话共享。

身份字段以 AES GCM 加密，用 HMAC SHA-256 支持唯一查重；当前示例共用一份应用字段密钥派生用途，生产应实施独立密钥、轮换与外部密钥管理。密钥只保存在本机 .local 配置中，不进入版本库或分发包。档案查询不返回身份密文、查重摘要或密码摘要。

所有用户值通过 JDBC 参数绑定。可选数据类别与存储过程名使用固定白名单，不能将用户传入的表名或 SQL 直接执行。角色权限在业务入口检查，家属关系、护工订单及机构辖区再决定行范围。服务仅绑定本机回环地址，若改为远程访问需增加 HTTPS 和部署侧访问限制。

备份与恢复

课程目标约定：每日全量，持续保留 binlog，每 15 分钟复制新增日志到独立备份位置，保留最近 7 天；RPO 目标 15 分钟，RTO 目标 30 分钟。这里的周期为部署方案，并未在用户电脑创建长期定时任务，RPO/RTO 也不是未经测量就承诺的实际性能。

全量使用 mysqldump --single-transaction --routines --triggers --events，保存结构、数据和例程。备份期间避免 DDL，记录与一致性快照对应的日志位置。实际部署应采用 dump 的日志位置信息或受控写入暂停来衔接全量与增量；不能随便把完成导出后的任意日志位置作为起点。演练脚本在受控课程实例上执行，演示单条事务的日志恢复，不等于覆盖所有生产故障。

恢复步骤：停止目标业务写入；在独立空库导入全量；按开始和结束日志位置重放 binlog；核对各基础表行数、外键和例程，再运行核心查询及业务拒绝测试；确认后再切换业务连接。禁止直接在未知的已有数据库中覆盖恢复。

实际证据见 evidence/recovery.json：全量恢复后重放一条真实的新增服务项目事务，并核对 35 张基础表数量。备份文件在 .local/backups，防止把含账号摘要和字段密文的数据随源码公开。

已遇到的问题与处理

1. 本机 Homebrew MySQL 的动态链接库缺失，Docker 内部磁盘也已满。最终复用可工作的 MySQL 8.4 程序，以独立数据目录和端口启动原生实例，不改动原实验数据库。

2. 初始表结构重复保存了计划所属老人和派单所属护工。根据函数依赖拆除重复字段，通过两个内部视图还原查询上下文，迁移时保留业务记录，然后重跑数据库集成测试。

3. 完整流程测试暴露了 JDBC 日期类型差异和直接访问被代理 DAO 字段的问题。分别统一时间对象处理、将访问封装回 DAO 方法，增加完整服务确认与工时复核的回归测试。

这些是真实开发问题。小组会议、成员协作及教师反馈须由真实参与人另行补充。

存储过程与触发器代码

以下核心例程按业务覆盖有效期、派单、工时、应急与补贴，全部审计和历史保护触发器在源代码附件中提供。

```
DELIMITER $$
CREATE PROCEDURE assert_plan(IN p_elder BIGINT,IN p_assess BIGINT,IN p_bundle BIGINT,IN p_start DATETIME,IN p_end DATETIME,IN p_id BIGINT)
BEGIN
 DECLARE guard_id BIGINT;
 SELECT id INTO guard_id FROM elder WHERE id=p_elder FOR UPDATE;
 IF NOT EXISTS(SELECT 1 FROM assessment a JOIN care_bundle b ON b.ability_level=a.ability_level WHERE a.id=p_assess AND a.elder_id=p_elder AND b.id=p_bundle AND a.assessed_at<=p_start AND a.valid_until>=p_end) THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='PLAN_ASSESSMENT_INVALID'; END IF;
 IF EXISTS(SELECT 1 FROM v_plan_context WHERE elder_id=p_elder AND id<>p_id AND status='ACTIVE' AND starts_at<p_end AND ends_at>p_start) THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='PLAN_OVERLAP'; END IF;
END$$
CREATE TRIGGER plan_before_insert BEFORE INSERT ON care_plan FOR EACH ROW BEGIN IF NEW.status='ACTIVE' THEN CALL assert_plan((SELECT elder_id FROM assessment WHERE id=NEW.assessment_id),NEW.assessment_id,NEW.bundle_id,NEW.starts_at,NEW.ends_at,0); END IF; END$$
CREATE TRIGGER plan_before_update BEFORE UPDATE ON care_plan FOR EACH ROW BEGIN IF NEW.status='ACTIVE' THEN CALL assert_plan((SELECT elder_id FROM assessment WHERE id=NEW.assessment_id),NEW.assessment_id,NEW.bundle_id,NEW.starts_at,NEW.ends_at,NEW.id); END IF; END$$
CREATE PROCEDURE expire_plans()
BEGIN
```

```
 UPDATE care_plan p JOIN assessment a ON a.id=p.assessment_id SET p.status=CASE WHEN p.ends_at<=NOW() THEN 'EXPIRED' ELSE 'PAUSED' END WHERE p.status='ACTIVE' AND (a.valid_until<=NOW() OR p.ends_at<=NOW());

END$$

CREATE PROCEDURE assert_shift(IN p_worker BIGINT,IN p_start DATETIME,IN p_end DATETIME,IN p_id BIGINT)

BEGIN

 DECLARE guard_id BIGINT;

 SELECT id INTO guard_id FROM caregiver WHERE id=p_worker FOR UPDATE;

 IF EXISTS(SELECT 1 FROM shift WHERE caregiver_id=p_worker AND id<>p_id AND status='OPEN' AND starts_at<p_end AND ends_at>p_start) THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='SHIFT_OVERLAP'; END IF;

END$$

CREATE TRIGGER shift_before_insert BEFORE INSERT ON shift FOR EACH ROW BEGIN CALL assert_shift(NEW.caregiver_id,NEW.starts_at,NEW.ends_at,0); END$$

CREATE TRIGGER shift_before_update BEFORE UPDATE ON shift FOR EACH ROW BEGIN IF NEW.status='OPEN' THEN CALL assert_shift(NEW.caregiver_id,NEW.starts_at,NEW.ends_at,NEW.id); END IF; END$$

CREATE PROCEDURE assert_order(IN p_elder BIGINT,IN p_item BIGINT,IN p_community BIGINT,IN p_start DATETIME,IN p_end DATETIME)

BEGIN

 IF NOT EXISTS(SELECT 1 FROM elder WHERE id=p_elder AND community_id=p_community AND status='ACTIVE') THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='ORDER_COMMUNITY_INVALID'; END IF;

 IF NOT EXISTS(SELECT 1 FROM v_plan_context p JOIN assessment a ON a.id=p.assessment_id JOIN plan_item i ON i.plan_id=p.id WHERE p.elder_id=p_elder AND i.item_id=p_item AND p.status='ACTIVE' AND p.starts_at<=p_start AND p.ends_at>=p_end AND a.valid_until>=p_end AND a.valid_until>NOW()) THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='ORDER_PLAN_INVALID'; END IF;

END$$
```

```
CREATE TRIGGER order_before_insert BEFORE INSERT ON service_order FOR EACH ROW BEGIN CALL assert_order(NEW.elder_id,NEW.item_id,NEW.community_id,NEW.starts_at,NEW.ends_at); END$$

CREATE TRIGGER order_before_update BEFORE UPDATE ON service_order FOR EACH ROW

BEGIN

 IF OLD.status='COMPLETED' THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='COMPLETED_ORDER_IMMUTABLE'; END IF;

 IF NEW.status='COMPLETED' AND NOT EXISTS(SELECT 1 FROM v_dispatch_context d JOIN service_record r ON r.dispatch_id=d.id WHERE d.order_id=NEW.id AND d.status='FINISHED' AND r.confirmed_by IS NOT NULL AND r.confirmed_at IS NOT NULL AND r.confirmation IS NOT NULL) THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='CONFIRMATION_REQUIRED'; END IF;

END$$

CREATE PROCEDURE assign_order(IN p_order BIGINT,IN p_shift BIGINT,IN p_reason VARCHAR(200))

BEGIN

 DECLARE v_elder,v_item,v_community,v_worker,v_guard,v_old_shift BIGINT;

 DECLARE v_start,v_end DATETIME;

 DECLARE v_state VARCHAR(30);

 SELECT elder_id,item_id,community_id,starts_at,ends_at,status INTO v_elder,v_item,v_community,v_start,v_end,v_state FROM service_order WHERE id=p_order FOR UPDATE;

 IF v_state IS NULL OR v_state NOT IN ('PENDING','ASSIGNED','ACCEPTED') THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='ORDER_NOT_DISPATCHABLE'; END IF;

 CALL assert_order(v_elder,v_item,v_community,v_start,v_end);

 SELECT caregiver_id INTO v_worker FROM shift WHERE id=p_shift;

 SELECT id INTO v_guard FROM caregiver WHERE id=v_worker FOR UPDATE;
```

```
 IF v_worker IS NULL OR NOT EXISTS(SELECT 1 FROM caregiver c JOIN caregiver_area a ON a.caregiver_id=c.id JOIN credential x ON x.caregiver_id=c.id WHERE c.id=v_worker AND c.status='ACTIVE' AND a.community_id=v_community AND x.item_id=v_item AND x.valid_from<=LEAST(NOW(),v_start) AND x.valid_until>=GREATEST(NOW(),v_end)) THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='WORKER_NOT_QUALIFIED'; END IF;

 IF EXISTS(SELECT 1 FROM inspection i WHERE i.status='OPEN' AND (i.caregiver_id=v_worker OR i.order_id IN(SELECT order_id FROM v_dispatch_context WHERE caregiver_id=v_worker))) THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='RECTIFICATION_REQUIRED'; END IF;

 IF EXISTS(SELECT 1 FROM v_dispatch_context d JOIN service_order o ON o.id=d.order_id WHERE d.caregiver_id=v_worker AND d.status IN ('WAITING','ACCEPTED') AND o.id<>p_order AND o.starts_at<v_end AND o.ends_at>v_start) THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='WORKER_TIME_CONFLICT'; END IF;

 SET v_old_shift=(SELECT shift_id FROM v_dispatch_context WHERE order_id=p_order AND active_order IS NOT NULL);

 IF v_old_shift IS NOT NULL THEN

  IF p_reason IS NULL OR LENGTH(TRIM(p_reason))<2 THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='REASSIGN_REASON_REQUIRED'; END IF;

  UPDATE dispatch SET status='REASSIGNED',reason=p_reason WHERE order_id=p_order AND active_order IS NOT NULL;

  UPDATE shift SET used_count=used_count-1 WHERE id=v_old_shift;

 END IF;

 UPDATE shift SET used_count=used_count+1 WHERE id=p_shift AND status='OPEN' AND starts_at<=v_start AND ends_at>=v_end AND used_count<capacity;

 IF ROW_COUNT()<>1 THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='SHIFT_FULL_OR_OUTSIDE'; END IF;

 INSERT INTO dispatch(order_id,shift_id,reason) VALUES(p_order,p_shift,COALESCE(p_reason,'首次派单'));

 UPDATE service_order SET status='ASSIGNED' WHERE id=p_order;
```

```
END$$

CREATE PROCEDURE verify_hours(IN p_dispatch BIGINT)

BEGIN

 UPDATE attendance a JOIN v_dispatch_context d ON d.id=a.dispatch_id JOIN
service_order o ON o.id=d.order_id SET a.status=IF(a.fence_ok AND
a.check_in>=o.starts_at-INTERVAL 30 MINUTE AND a.check_out<=o.ends_at+INTERVAL
30 MINUTE,'VALID','REVIEW'),a.effective_minutes=IF(a.fence_ok AND
a.check_in>=o.starts_at-INTERVAL 30 MINUTE AND a.check_out<=o.ends_at+INTERVAL
30 MINUTE,TIMESTAMPDIFF(MINUTE,a.check_in,a.check_out),0) WHERE
a.dispatch_id=p_dispatch AND a.check_out IS NOT NULL AND a.status='PENDING';

END$$

CREATE TRIGGER vital_before_insert BEFORE INSERT ON vital FOR EACH ROW

BEGIN

 DECLARE v_low,v_high DECIMAL(12,3);

 IF NOT EXISTS(SELECT 1 FROM device_binding WHERE id=NEW.binding_id AND
bound_at<=NEW.collected_at AND (unbound_at IS NULL OR
unbound_at>NEW.collected_at)) THEN SIGNAL SQLSTATE '45000' SET
MESSAGE_TEXT='VITAL_BINDING_TIME_INVALID'; END IF;

 SELECT low_value,high_value INTO v_low,v_high FROM metric_rule WHERE
id=NEW.metric_rule_id;

 SET NEW.abnormal=(NEW.metric_value<v_low OR NEW.metric_value>v_high);

END$$

CREATE TRIGGER vital_after_insert AFTER INSERT ON vital FOR EACH ROW

BEGIN

 IF NEW.abnormal THEN

 INSERT INTO emergency(elder_id,vital_id,source,deadline,status) SELECT
b.elder_id,NEW.id,'DEVICE',DATE_ADD(NOW(),INTERVAL m.response_seconds
SECOND),'UNCONFIRMED' FROM device_binding b JOIN metric_rule m ON
m.id=NEW.metric_rule_id WHERE b.id=NEW.binding_id;

 END IF;
```

```
END$$

CREATE PROCEDURE escalate_events()

BEGIN

 INSERT IGNORE INTO emergency_log(event_id,level,message) SELECT id,level+1,'响应超时，升级至值班主管' FROM emergency WHERE status IN('OPEN','UNCONFIRMED') AND deadline<=NOW() AND level<3;

 UPDATE emergency SET level=level+1,deadline=DATE_ADD(NOW(),INTERVAL 5 MINUTE) WHERE status IN('OPEN','UNCONFIRMED') AND deadline<=NOW() AND level<3;

 INSERT IGNORE INTO notification(event_id,account_id,message) SELECT e.id,a.id,CONCAT('应急事件待处理，级别 ',e.level) FROM emergency e JOIN elder x ON x.id=e.elder_id JOIN account a ON a.community_id=x.community_id AND a.role IN('ADMIN','REGULATOR') WHERE e.status IN('OPEN','UNCONFIRMED');

END$$

CREATE PROCEDURE submit_claim(IN p_elder BIGINT,IN p_policy BIGINT,IN p_period CHAR(7),IN p_orders JSON)

BEGIN

 DECLARE v_guard,v_item,v_claim,v_order BIGINT;

 DECLARE v_rate,v_amount,v_price DECIMAL(12,2);

 DECLARE v_minutes,v_i INT DEFAULT 0;

 DECLARE v_unit VARCHAR(10);

 SELECT id INTO v_guard FROM elder WHERE id=p_elder FOR UPDATE;

 SELECT item_id,rate INTO v_item,v_rate FROM policy WHERE id=p_policy;

 IF v_item IS NULL OR JSON_LENGTH(p_orders)=0 OR p_period NOT REGEXP '^[0-9]{4}-(0[1-9]|1[0-2])$' THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='CLAIM_INPUT_INVALID'; END IF;

 INSERT INTO claim(elder_id,item_id,period) VALUES(p_elder,v_item,p_period);

 SET v_claim=LAST_INSERT_ID();

 WHILE v_i<JSON_LENGTH(p_orders) DO
```

```
 SET v_order=CAST(JSON_UNQUOTE(JSON_EXTRACT(p_orders,CONCAT('$[',v_i,']')))
AS UNSIGNED);

 IF NOT EXISTS(SELECT 1 FROM service_order o JOIN policy p ON p.id=p_policy JOIN
elder e ON e.id=o.elder_id JOIN service_item s ON s.id=o.item_id WHERE o.id=v_order
AND o.elder_id=p_elder AND o.item_id=v_item AND o.status='COMPLETED' AND
o.community_id=p.community_id AND o.starts_at>=p.starts_at AND o.ends_at<=p.ends_at
AND DATE_FORMAT(o.starts_at,'%Y-%m')=p_period AND s.subsidizable=1 AND
EXISTS(SELECT 1 FROM assessment a WHERE a.elder_id=e.id AND
a.ability_level=p.ability_level AND a.assessed_at<=o.starts_at AND
a.valid_until>=o.ends_at)) THEN SIGNAL SQLSTATE '45000' SET
MESSAGE_TEXT='CLAIM_ORDER_INELIGIBLE'; END IF;

 SET v_minutes=(SELECT a.effective_minutes FROM attendance a JOIN
v_dispatch_context d ON d.id=a.dispatch_id JOIN service_record r ON r.dispatch_id=d.id
WHERE d.order_id=v_order AND d.status='FINISHED' AND a.status='VALID' AND
r.confirmed_by IS NOT NULL LIMIT 1);

 IF v_minutes IS NULL OR v_minutes<=0 THEN SIGNAL SQLSTATE '45000' SET
MESSAGE_TEXT='CLAIM_EVIDENCE_MISSING'; END IF;

 SELECT o.amount,s.unit INTO v_price,v_unit FROM service_order o JOIN service_item s
ON s.id=o.item_id WHERE o.id=v_order;

 SET v_amount=LEAST(v_price,ROUND(IF(v_unit='HOUR',v_minutes/60*v_rate,v_rate),2));

 INSERT INTO
claim_item(claim_id,policy_id,order_id,amount,effective_minutes,rate_snapshot)
VALUES(v_claim,p_policy,v_order,v_amount,v_minutes,v_rate);

 SET v_i=v_i+1;

 END WHILE;

 UPDATE claim SET amount=(SELECT SUM(amount) FROM claim_item WHERE
claim_id=v_claim),status='SUBMITTED' WHERE id=v_claim;

END$$

CREATE TRIGGER claim_before_update BEFORE UPDATE ON claim FOR EACH ROW

BEGIN
```

```
 IF OLD.status='PAID' THEN SIGNAL SQLSTATE '45000' SET
MESSAGE_TEXT='PAID_CLAIM_IMMUTABLE'; END IF;

 IF NEW.status<>OLD.status AND NOT ((OLD.status='DRAFT' AND
NEW.status='SUBMITTED') OR (OLD.status='SUBMITTED' AND NEW.status
IN('STREET_APPROVED','REJECTED')) OR (OLD.status='STREET_APPROVED' AND
NEW.status IN('APPROVED','REJECTED')) OR (OLD.status='APPROVED' AND
NEW.status='PAID')) THEN SIGNAL SQLSTATE '45000' SET
MESSAGE_TEXT='CLAIM_STATE_INVALID'; END IF;

 IF NEW.status<>'DRAFT' AND NEW.amount<>(SELECT COALESCE(SUM(amount),0)
FROM claim_item WHERE claim_id=NEW.id) THEN SIGNAL SQLSTATE '45000' SET
MESSAGE_TEXT='CLAIM_AMOUNT_MISMATCH'; END IF;

END$$

DELIMITER ;

DELIMITER $$
```

测试与结果

Tests run: 38, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 3.894 s -- in

edu.bjfu.eldercare.DatabaseIntegrationTest

测试连接独立的真实 MySQL 测试库。覆盖全部表最低记录数、15 组查询等价、视图执行、外键与唯一性、有效期、重叠区间、派单容量、确认凭据、工时复核、设备历史、异常事件、补贴回滚、身份隔离、登录锁定、会话退出及并发。完整方法名和原始结果保存在 Surefire XML 与测试源代码中。

验证对象	实际验证内容
并发计划	两个事务同时创建同老人重叠计划，只成功一个
并发派单	两个订单竞争最后一个班次名额，只成功一个
失败改派	新班次不覆盖预约时刻时，旧派单和名额被回滚恢复
批量写入	一条合法、一条越权的数据批次整体回滚

验证对象	实际验证内容
完整服务	接单、签到、异常工时复核、签退、家属确认走通
备份恢复	全量加实际 binlog 增量在独立库恢复，35 表计数一致

七、附件内容

附件编号	内容与位置
01	docs/diagrams：全局与局部 UML、用例图、鲁棒图，含 DOT、SVG、PNG
02	database：全部 DDL、视图、过程、触发器、15 组双实现查询
03	src 与 pom.xml：Java 源码、静态界面和测试源码
04	docs/数据字典与关系模式.md：所有字段的名称、类型、默认值、关联与含义
05	evidence：测试日志、查询执行计划、行数、恢复演练、浏览器截图
06	eldercare.jar 与 Windows 初始化、启动脚本
07	答辩演示文稿与 5 分钟录制讲稿；全员发言视频待真实录制
08	个人模板预填稿；个人实际分工和贡献待本人核对
09	教师提问反馈与解决方案待答辩发生后补充

提交前应核对组名和成员名单、真实分工、个人报告与答辩的一致性，并补齐全员发言的录像及真实协作证据。不得把预设问答当作已发生的教师提问。

参考资料

课程设计任务书 2026，第 1—9 页。

教师任务讲解逐字稿，00:01:58—00:34:23。

数据库系统课程设计报告模板 2026 小组与个人版。

数据库系统课程设计评分标准 2026 版。

MySQL 官方文档：CREATE TRIGGER、CREATE PROCEDURE、SIGNAL，https://dev.mysql.com/doc/refman/8.0/en/create-trigger.html 。

Spring 官方 JDBC 文档：https://docs.spring.io/spring-framework/reference/data-access/jdbc/core.html 。